

Eclipse Soundscapes Data Management Public Data Sharing (Stage 3)

A methods and provenance reference for people using Eclipse Soundscapes (ES) datasets

Authors/Creators

Henry “Trae” Winter
ORCID: [0000-0002-6678-590X](https://orcid.org/0000-0002-6678-590X)

MaryKay Severino
ORCID: [0000-0002-2363-7421](https://orcid.org/0000-0002-2363-7421)

Affiliation

ARISA Lab, L.L.C.

Funding

NASA Science Activation Award 80NSSC21M0008

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the National Aeronautics and Space Administration (NASA).

DOI

<https://doi.org/10.5281/zenodo.18683437>

Table of Contents

Table of Contents	2
1. What this Document Covers	4
2. What Stage 3 produced	4
3. Why Stage 3 Was Needed	5
4. Data Platform Strategy and the Transition to Zenodo	6
4.1 Arbimon Platform Alignment	6
4.2 NASA DAAC Evaluation	7
4.3 Adoption of Zenodo as Primary Archive	7
5. Dataset Bundling and Packaging	8
5.1 Raw Audio Preservation and Metadata Provenance	8
5.2 ZIP Archive Structure	8
5.3 Supplementary Files	9
5.4 File List and SHA-512 Hashes	10
6. Metadata Templates and Standardization	10
6.1 Project Identity: Project_config.json	10
6.2 README Template	10
6.3 Data Dictionaries	10
6.4 Citations: Related Identifiers and References	11
7. Geographic Descriptors and Eclipse Parameters	11
8. Stage 3 Data Preparation and Publication Workflow	13
8.1 Architecture	13
8.2 Data Preparation Workflow	13
8.2.1 Template Creation for Zenodo Records	13
8.2.2 HTML Template Configuration for Zenodo Records	14
8.2.3 Development of Zenodo-Specific Metadata Spreadsheets	14
8.2.4 Configuration of Project Metadata and Upload Settings (JSON)	15
8.2.5 Automated Dataset Assembly (AZUS Execution)	16
8.2.6 Output Packaging and Site-Level Record Structure	17
8.2.7 Final Output: Zenodo Records	18
9. DOI Creation and Versioning	19
10. Upload Verification and Integrity	19
10.1 SHA-512 file hashing	19
10.2 Upload Tracking and Result Logging	19
10.3 Metadata Completeness	20
10.4 Request Logging	20
11. Validation and Error Handling	20
11.1 Metadata Completeness Review	20
11.2 ES ID # Alignment Validation	20

11.3 Manual Intervention for Flagged Inconsistencies	20
11.4 Dry-Run Mode	21
12. What These Procedures Support	21
13. Open-Source Tooling and Generalization	21
14. Alignment with FAIR Principles	22
14.1 Findable	22
14.2 Accessible	22
14.3 Interoperable	22
14.4 Reusable	23
15. Glossary	24
16. References and Related Resources	28
Core Eclipse Soundscapes Resources	28
Eclipse Soundscapes Technical Workflow References	28
Eclipse Timing and Astronomical Resources	28
AudioMoth and Acoustic Recording Resources and References	28
Soundscape Ecology and Acoustic Science References	29
Open Science and Repository Infrastructure Resources	29
Supporting Technical and Infrastructure References	29
Appendix 1: ES Operational Infrastructure and Data Stewardship Workflows	30
Appendix 2: Upload Command Line Interface (CLI)	31
Appendix 3: AZUS Command-Line Interface and Execution	32
A3.1 Overview	32
A3.2 Execution Environment and Setup	32
A3.3 Batch Upload Execution Commands	32
A3.4 Upload Workflow Behavior	32
A3.5 Validation and User Interaction	32
A3.6 Logging and Output Tracking	33
A3.7 Configuration Dependence	33

1. What this Document Covers

This document describes Stage 3 (Public Data Sharing) of the Eclipse Soundscapes (ES) data lifecycle. It explains how validated site-level records were packaged into structured datasets, assigned persistent identifiers, and published to the Zenodo open-data repository for long-term preservation, public access, and scientific reuse.

Eclipse Soundscapes is a NASA-funded participatory science project that deployed AudioMoth acoustic recording devices during two eclipses: the October 14, 2023 annular solar eclipse and the April 8, 2024 total solar eclipse. Volunteers placed recorders across the United States to capture how rapid eclipse-related changes in light affected animal behavior and environmental soundscapes.

Stage 3 transformed validated site-level records produced during Stages 1 and 2 into publicly archived, DOI-assigned datasets. This stage established the long-term preservation and open-science framework for Eclipse Soundscapes by packaging audio data, metadata, documentation, and supporting resources into standardized archival records published through the Eclipse Soundscapes Zenodo Community. These workflows supported dataset discoverability, citation, reproducibility, and long-term public reuse.

Specifically, this document details:

- preparation and packaging of validated site-level datasets
- ZIP archive structure and companion documentation
- metadata preparation and README generation
- Automated Zenodo Upload Software (AZUS) workflows
- Zenodo API integration and DOI minting
- upload verification, integrity checks, and community review workflows

This document is intended as a methods and provenance reference for researchers, archivists, repository managers, developers, and participatory science practitioners interested in understanding how large-scale community-generated datasets were prepared for long-term open-data preservation and public scientific reuse.

2. What Stage 3 produced

Stage 3 transformed validated site-level datasets from Stage 2 into publicly accessible, DOI-assigned archival records designed for long-term preservation, public access, and scientific reuse.

- **Public Zenodo Dataset Records**
Site-level Zenodo records containing raw audio data, metadata, documentation, data dictionaries, and integrity verification files organized by ES ID #.
- **The Eclipse Soundscapes Zenodo Community**
A curated public repository organizing Eclipse Soundscapes datasets, documentation, and related resources into a centralized discovery and long-term preservation framework.
- **Persistent DOI Assignment**
Unique Data site DOIs are assigned to each site-level dataset, supporting citation, traceability, and long-term discoverability within the scientific research ecosystem.
- **Open-Source Archival and Publication Software (AZUS)**
Development and operational deployment of the Automated Zenodo Upload Software

(AZUS), a configuration-driven batch publication system supporting scalable dataset packaging, metadata generation, DOI assignment, and Zenodo API uploads.

- **Standardized Archival Dataset Packages**

Self-contained ZIP-based archival dataset structures containing raw audio recordings, metadata files, README documentation, data dictionaries, integrity manifests, and supporting reference materials.

These deliverables established the long-term open-science preservation and public data-sharing framework for the Eclipse Soundscapes project while supporting reproducibility, accessibility, and scientific reuse of the datasets.

3. Why Stage 3 Was Needed

Stage 3 was necessary because validated Eclipse Soundscapes datasets required a long-term public preservation and access framework that supported open science, dataset citation, reproducibility, and large-scale public reuse.

The project generated hundreds of site-level audio datasets distributed across two eclipse campaigns. These datasets needed to be:

- preserved in stable archival systems,
- organized into consistent public records,
- associated with standardized metadata,
- assigned persistent identifiers,
- and made publicly accessible without exposing participant identities.

Because no existing workflow supported efficient publication of hundreds of site-level datasets, the project also required scalable tooling for automated dataset preparation, metadata generation, and repository upload.

Stage 3 established the long-term public data stewardship and open-science infrastructure for the Eclipse Soundscapes project.

The workflow diagram below provides a Stage 3–focused view of the Eclipse Soundscapes data lifecycle, showing only the directly relevant deliverables from related stages. The complete ES Operational Infrastructure and Data Stewardship Workflows diagram is included in [Appendix 1](#).

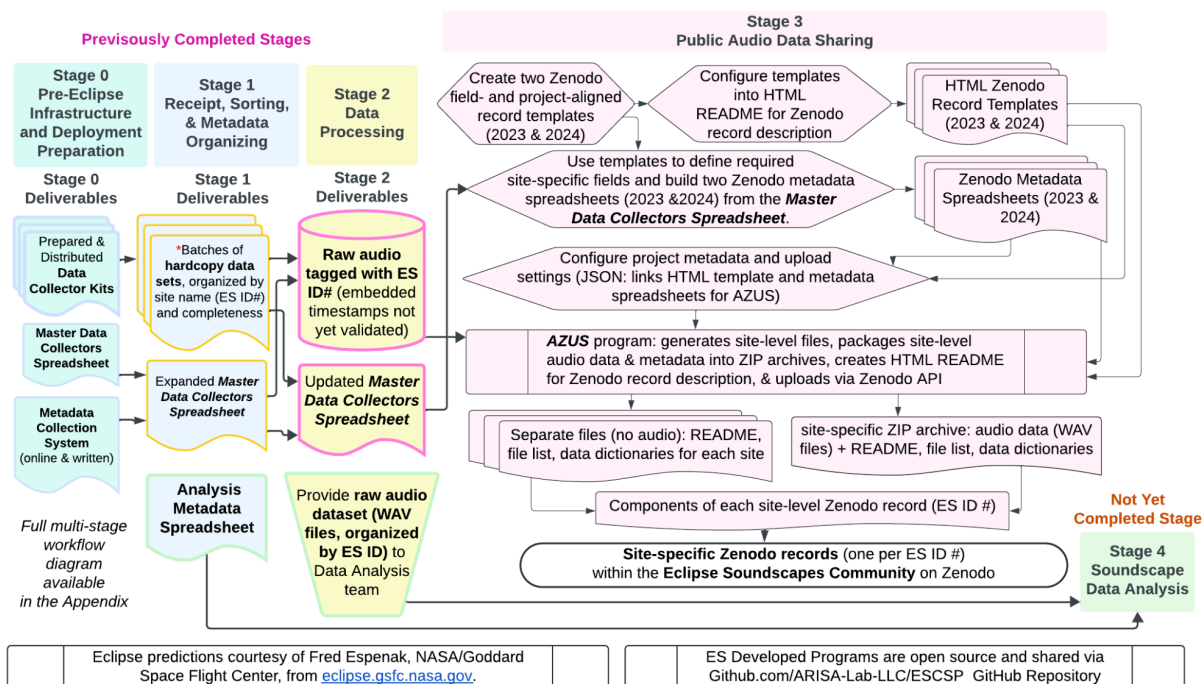


Figure 1. Eclipse Soundscapes Stage 3 workflow for packaging, documenting, and publicly archiving validated audio datasets through the Eclipse Soundscapes Zenodo Community. The workflow illustrates how site-level audio data, metadata spreadsheets, README templates, and integrity documentation were standardized and processed through the Automated Zenodo Upload Software (AZUS) to generate DOI-assigned Zenodo records supporting long-term preservation, open-science stewardship, public access, and scientific reuse.

4. Data Platform Strategy and the Transition to Zenodo

The original data management strategy for Eclipse Soundscapes specified Arbimon as the primary platform for data storage, browsing, and analysis, with an ARISA Lab-managed cloud server supporting raw data downloads and Zenodo serving as a secondary platform for DOI assignment and citation. This architecture was redesigned prior to the 2023 and 2024 eclipse deployments due to changes in platform alignment, long-term access requirements, and data stewardship considerations.

4.1 Arbimon Platform Alignment

The initial data management approach relied on Arbimon, a platform designed primarily for acoustic data analysis. Arbimon also supported the upload and download of audio data and provided open APIs, which made it possible to use the platform for data storage while giving Eclipse Soundscapes participants direct access to built-in analysis tools. The Arbimon team expressed interest in supporting this type of integration, making it a suitable choice during early project planning.

Following Arbimon's acquisition by Rainforest Connection (RFCx), the platform entered a period of organizational and technical transition. During this time, leadership, system development, and APIs were evolving as part of the broader transition. As a result, the features needed for long-term data management, particularly stable APIs and consistent access to upload and download functionality, became less predictable.

Because the Eclipse Soundscapes project required a system with a primary and sustained focus on long-term data storage, open access, and reliable programmatic access, the team determined that a repository-centered solution would better support its long-term data stewardship needs.

4.2 NASA DAAC Evaluation

The project team evaluated the use of NASA Distributed Active Archive Centers (DAACs) as a potential long-term data repository. DAACs provide well-established infrastructure for storing, managing, and distributing scientific datasets, particularly those associated with NASA missions.

However, DAACs are organized around specific science domains and mission-based data products. It was determined that no DAAC was assigned to the NASA Science Activation program or to citizen science datasets of this type. Direct engagement with the Solar Data Analysis Center (SDAC) confirmed that existing systems were designed to support data from established NASA and ESA missions and were not structured to accommodate large collections of site-based audio recordings generated through a participatory science project.

Supporting the Eclipse Soundscapes dataset within a DAAC would have required substantial changes to existing systems, which were not feasible within the current operational scope. As a result, the DAAC option was not pursued further.

4.3 Adoption of Zenodo as Primary Archive

With Arbimon no longer meeting the project's needs and no suitable NASA archive available, the project adopted Zenodo (zenodo.org) as the primary repository for all Eclipse Soundscapes datasets. Zenodo is a free, open-access research data repository operated by CERN that provides long-term data preservation, persistent DOI assignment, and unrestricted data download across scientific domains.

Using Zenodo required a change in how the data were organized. While the total dataset was too large to store as a single record, individual site-level recordings were well within Zenodo's file size limits (50 GB per record). To accommodate this, each recording location—identified by a unique Eclipse Soundscapes ID number (ES ID #)—was published as its own Zenodo record. Additional records were created for training materials, documentation, and methodological references.

This site-level structure made it possible to:

- assign a DOI to each recording site
- allow users to find and download data from specific locations
- enable individual data collectors to locate and access recordings and associated metadata from their own recording site
- scale the archive across hundreds of sites

Publishing data in this way also introduced a practical challenge. With hundreds of recording sites, manually creating and uploading each Zenodo record would have been time-consuming and difficult to manage consistently.

To address this, the Eclipse Soundscapes team developed the Automated Zenodo Upload Software (AZUS). AZUS is a configuration-driven system that prepares and uploads datasets in batches, ensuring that each record includes consistent metadata, documentation, and file structure. This approach was made possible by Zenodo's well-documented, open API, which

supports automated interaction with the repository and enables scalable data publication workflows.

Together, the use of Zenodo and the development of AZUS provided a stable, open, and scalable system for publishing and preserving Eclipse Soundscapes data.

5. Dataset Bundling and Packaging

This section describes the structure and contents of each dataset, while the assembly and packaging process is detailed in Section 6. Each ES ID # dataset is distributed as a complete, self-contained package within a lossless ZIP archive. Python scripts within the Resources/ folder within AZUS automate the entire process of taking a directory of raw data files and bundling them within the ZIP archive, along with supplementary files, as well as creating site-specific metadata files. A detailed description of the ZIP archive's structure and its contents is provided in the following section.

It was decided to include only raw audio data files in each site-specific ZIP archive and Zenodo record. This decision was so that any researcher can independently verify, reproduce, and extend the analysis performed. As a result, some sites have WAV files with 0 bytes of data or timestamps outside the range of probable recording times. Procedures used by the Eclipse Soundscapes team to process audio data for its purposes are outlined in the Data Management reports located in the Eclipse Soundscapes Zenodo community. Data with 0 bytes of data were included for completeness.

5.1 Raw Audio Preservation and Metadata Provenance

The WAV audio files shared through Zenodo are byte-for-byte identical to the contents of the returned microSD cards. No timestamp corrections, audio processing, resampling, or other modifications were applied to the shared audio data at any stage of the workflow.

The metadata records accompanying each dataset do reflect the validation and reconciliation work completed during Stage 2. These metadata include ES ID # assignments, geographic coordinates, eclipse timing parameters, and documentation of timestamp state derived through the Stage 2 processing workflow.

This separation between raw audio data and documented metadata is intentional. It allows the public archive to support both researchers who want to work directly from the original, unprocessed recordings and those who prefer to begin from the metadata annotations and validation context produced during Stage 2.

5.2 ZIP Archive Structure

All WAV audio files and the AudioMoth CONFIG.TXT configuration file are compressed into a single ZIP archive named ES ID_XXX.zip, where the XXX represents the three-digit ES ID number that uniquely identifies the site of the audio recording. Audio files are stored byte-for-byte identically to the original microSD card contents. No audio processing, resampling, or timestamp correction is applied to the files at any stage. ZIP uses DEFLATED compression, which allows for lossless compression of multiple files and their file structure. Upon the decompression of the ZIP file, a folder following the ES ID_XXX/ naming convention. In addition to the CONFIG.TXT and WAV files, all supplementary files described in the next section are included in the ZIP archive.

5.3 Supplementary Files

Each Zenodo record includes the following companion files, which are also included with the ZIP archive.

File	Purpose
README.html / README.md	Human-readable documentation with site location, eclipse parameters, recording dates, and usage guidelines. Generated from an HTML template with per-site variable substitution.
total_eclipse_data.csv	Single-row comma-separated variable (CSV) file with machine-readable metadata for this site: ES ID #, coordinates, eclipse type, timing data, and version.
file_list.csv	Inventory of every file in the record with file type, description, size, SHA-512 hash, and associated data dictionary.
CONFIG.TXT	AudioMoth device configuration file (also included inside the ZIP). Provided separately for convenient access without extracting the archive.
Data dictionaries (4 CSV files)	One data dictionary for each structured data file: WAV_data_dict.csv, CONFIG_data_dict.csv, file_list_data_dict.csv, and 2024_total_eclipse_data_data_dict.csv.
License.txt	Creative Commons Attribution 4.0 International (CC BY-SA 4.0) license text.
AudioMoth_Operation_Manual.pdf	AudioMoth device operation manual for reference.
Eclipse Soundscapes Data Collector Role Training and Implementation Manual (2023–2024) (For Zenodo archiving)	A PDF version of the training that Data Collectors went through to learn how to collect soundscape data using an AudioMoth device.
Eclipse Soundscapes Data Management Data Processing (Stage 2)	A PDF document that describes how the data was submitted, cataloged, and processed before preparation to upload to Zenodo
Eclipse Soundscapes Data Management Public Data Sharing (Stage 3)	This PDF document, which describes how the data was publicly shared.

Table 1. Companion files are included in each Zenodo record.

5.4 File List and SHA-512 Hashes

The `file_list.csv` serves as the manifest for each record. It lists every file, including individual WAV recordings, their sizes, and a SHA-512 cryptographic hash computed at packaging time. The SHA-512 hash function was chosen because it is a widely trusted integrity fingerprint that reduces the risk of accidental hash collisions that can occur in older hash functions. These traits allow downstream users to verify data integrity after download with confidence. The `file_list.csv` is created last in the preparation sequence because it must hash all other files, including in the ZIP archive. The `file_list.csv` file is the only file that does not have a SHA-512 hash calculated, as that would cause a recursive error.

6. Metadata Templates and Standardization

This section describes the standardized metadata structures and configuration components used across all records. Their role within the Stage 3 workflow is described in Section 8. A central design principle of AZUS is that all project-specific identity and metadata are externalized to human-readable configuration files. No Eclipse Soundscapes-specific data is hardcoded in Python, and all project-specific files live within the *Resources/* subfolder within AZUS, and any project-specific code or scripts live within the *Scripts/* subfolder.

6.1 Project Identity: `Project_config.json`

The `project_config.json` file defines all project-level metadata that applies to every record: creators (with ORCID identifiers and roles), contributors, funding information (including NASA award number), Zenodo community ID, custom fields, license, resource type, and CSV file header expectations. Adapting AZUS for a different citizen science project requires the editing of this file.

6.2 README Template

Each Zenodo record includes a *Markdown-formatted README.md* file that serves as the basis for the HTML format file for the Zenodo record description, *README.html*. Both files are generated from an HTML template (*README_template.html*) using Python's string. Template substitution. The template contains *\$variable* placeholders that are replaced with site-specific values at preparation time: ES ID number, geographic coordinates, eclipse type, contact times, recording dates, and version information. This approach ensures consistent documentation language across hundreds of records while allowing per-site customization. The use of an HTML file as a template allows other projects to create their own data-specific *README* files and Zenodo descriptions with relative ease.

6.3 Data Dictionaries

Four data dictionary CSV files are included with every record. Each defines the variable name, description, data type, units, size, source, code meanings, and notes for a corresponding data file. The dictionaries cover: WAV audio file header fields, *CONFIG.TXT* device settings, *file_list.csv* inventory columns, and the site-specific metadata file, *total_eclipse_data.csv*. These dictionaries are the same for each data site and are maintained as shared resources in the

Resources/ subfolder within the AZUS directory tree. At data preparation time, these files are copied into every staging directory to be included in the Zenodo upload and ZIP archive.

6.4 Citations: Related Identifiers and References

Each Zenodo record includes structured citation metadata linking it to related resources. The *related_identifiers.csv* file lists DOIs with their relationship type (e.g., “is supplemented by”) and resource type (video, report). The *references.csv* file lists free-text bibliographic references. Both files support a per-record override mechanism: if a *related_identifiers.csv* or *references.csv* exists inside an ES ID # staging directory, AZUS uses that file instead of the global default located in *Resources/*. This allows specific datasets to carry additional or different citations without affecting the entire batch.

7. Geographic Descriptors and Eclipse Parameters

Metadata for each record is compiled from a master data collectors spreadsheet, which contains geographic and eclipse-specific details for every data collection site, as identified by a unique Eclipse Soundscapes ID number. Two separate master spreadsheets are utilized depending on the event: *2023_Annular_Zenodo_Form_Spreadsheet.csv* for the 2023 Annular Solar Eclipse, and *2024_Total_Zenodo_Form_Spreadsheet.csv* for the 2024 Total Solar Eclipse.

For each site, the metadata from the master data collectors spreadsheet is used to create a single row, machine-readable *total_eclipse_data.csv* spreadsheet for that site. Site-specific data is also embedded in both the *README.md* and *README.html* files. The information for each site is summarized in Table 2.

Field	Description
Latitude / Longitude	Decimal degree coordinates of the recording site. Provided by volunteers at registration, verified during Stage 2.
Local Eclipse Type	Annular, total, or partial, based on the site's position relative to the eclipse path.
Eclipse Percent (%)	The maximum percentage of the sun obscured at the site location.
Eclipse contact times (UTC)	Start (1st contact), totality start (2nd), maximum, totality end (3rd), and end (4th contact). Partial and annular sites omit 2nd/3rd contact times.
Eclipse date	Calendar date of the eclipse event.
WAV file time/date settings	Whether the AudioMoth clock was set correctly was noted by the volunteer.
Recording date range	First and last recording dates, extracted from WAV filenames during upload.
Keywords and subjects	Zenodo subject tags include eclipse type, location descriptors, and scientific context.

Table 2. Geographic and eclipse parameters are included in each record.

Timing data for eclipse contact times was computed using predictions from NASA Goddard Space Flight Center (eclipse.gsfc.nasa.gov), and was applied per-site based on geographic coordinates. This ensures each record carries the correct local eclipse window for its specific location. The code used for the calculation of eclipse timings as a function of location can be found in the *Eclipse Look-Up Tool* and the *Eclipse Phase Timing Tool (EPTT)* sections of the Eclipse Soundscapes GitHub repository (github.com/ARISA-Lab-LLC/ESCSP).

These structured metadata inputs are used during the Stage 3 workflow to populate standardized Zenodo records, as described in Section 8.

8. Stage 3 Data Preparation and Publication Workflow

The Automated Zenodo Upload Software (AZUS) is a Python-based system designed to automate the batch upload of site-level datasets to the Zenodo repository in a manner accessible to researchers and citizen science project teams. The setup and use of the AZUS codebase is covered in detail in the *README.md* file contained in the *ESCSP-AZUS* tree of the Eclipse Soundscapes GitHub repository (github.com/ARISA-Lab-LLC/ESCSP). The updated installation instructions of the AZUS virtual environment and other steps in the codebase setup can be found there and will not be covered in this document.

8.1 Architecture

AZUS communicates with Zenodo through its InvenioRDM REST API. The upload pipeline for Eclipse Soundscapes is implemented across three Python modules, each handling a distinct responsibility:

Module	Responsibility
<code>prepare_dataset.py</code>	Project-specific Python code that bundles raw data into upload-ready staging directories. Handles ZIP creation, README generation, site-specific files, file hashing, and manifest creation.
<code>standalone_tasks.py</code>	Orchestrates the upload pipeline. Discovers datasets, parses collector CSV files, constructs Zenodo metadata payloads (InvenioRDM schema), and manages batch execution with progress tracking and result logging.
<code>standalone_uploader.py</code>	Low-level Zenodo API client. Handles HTTP communication: draft creation, file upload, community submission, and optional publishing. All functions are synchronous with no external orchestration dependencies.

Table 3. AZUS module architecture.

8.2 Data Preparation Workflow

The Stage 3 data preparation workflow transforms validated site-level records into structured, upload-ready datasets through a configuration-driven, stepwise processing pipeline. Initial steps within Stage 3 establish project templates, metadata structures, and configuration files that define how site-level data are organized. Dataset assembly is then executed by the Python script *prepare_dataset.py* within the AZUS framework, as reflected in the Stage 3 panel of the workflow diagram (Figure 1).

8.2.1 Template Creation for Zenodo Records

Stage 3 begins with the development of standardized Zenodo record templates that define the full structure, content, and documentation requirements for each site-level dataset.

- Two templates were created (2023 annular and 2024 total solar eclipse), reflecting differences in eclipse type and associated metadata fields

- Templates were developed through careful review of Zenodo's metadata schema and capabilities to determine what a complete, reusable, and well-documented dataset should include
- Each template defines:
 - required metadata fields (e.g., creators, roles, funding, licensing, related works)
 - site-specific scientific attributes (e.g., location, eclipse timing, percent coverage)
 - standardized dataset descriptions and contextual project information
 - citation formats and data versioning conventions

In addition to metadata structure, the templates incorporate detailed documentation of data collection methods, timestamp protocols, and data processing workflows (Stages 1–3), ensuring that each record is self-contained and reproducible.

These templates serve as both:

- A **data model** for structuring site-level records and defining required metadata inputs
- and a **documentation framework** that ensures consistency, transparency, and long-term usability across all Zenodo records

This approach ensured that each ES ID # record functions as a complete, standalone dataset with sufficient metadata and contextual information to support independent interpretation and reuse.

8.2.2 HTML Template Configuration for Zenodo Records

Following the development of Zenodo record templates, these structures were implemented as reusable HTML templates to support consistent, site-level record generation.

Initial versions of the templates, described in the previous section, were developed in a collaborative, human-readable format to support iterative team review and refinement of content and structure before being translated into structured HTML for programmatic use.

- A standardized **HTML README template** was created to represent the full Zenodo record description
- The template incorporates all required fields defined in Section 7, including:
 - dataset description and contextual project information
 - site-specific scientific attributes (e.g., location, eclipse timing)
 - data collection and processing documentation
 - citation, licensing, and related works

The HTML format was selected to:

- preserve formatting and readability within Zenodo records
- support structured, human-readable documentation
- enable consistent rendering across all site-level datasets

The HTML template includes variable placeholders corresponding to site-specific metadata fields. These placeholders are designed to be dynamically populated during dataset assembly using values from the Zenodo metadata spreadsheets.

This step operationalizes the template structure by transforming it into a format that can be programmatically populated and deployed across all ES ID # records.

8.2.3 Development of Zenodo-Specific Metadata Spreadsheets

Following template development, site-level metadata were structured into standardized Zenodo metadata spreadsheets designed to populate the variable fields defined in the record templates.

- Two metadata spreadsheets were created (2023 annular and 2024 total solar eclipse)
- These spreadsheets were **derived from the *Master Data Collectors Spreadsheet***, which served as the comprehensive internal record of all site-level information

The Master Data Collectors Spreadsheet contained significantly more detailed information per site, including internal-use fields and personally identifiable information (PII) that were not appropriate for public release. As a result, a new set of **Zenodo metadata spreadsheets** was created to:

- extract only the fields required for public data sharing
- align directly with the variable fields defined in the Zenodo record templates
- standardize formatting for consistent downstream processing

Each row in the Zenodo metadata spreadsheet corresponds to a single ES ID # and provides the site-specific values needed to populate:

- dataset titles and descriptions
- geographic coordinates
- eclipse parameters (type, timing, percent coverage)
- recording date ranges and timestamp context
- contributor roles and affiliations (as appropriate for public release)

In this way, the metadata spreadsheets function as a curated, publication-ready subset of the master dataset, ensuring that all publicly shared records:

- exclude sensitive or private information
- maintain consistency with Zenodo metadata requirements
- provide complete and accurate site-level context

These spreadsheets operate in direct coordination with the templates: the templates define the structure and required fields, and the metadata spreadsheets supply the site-specific values.

Together, they form a complementary system in which standardized record structures are dynamically populated with curated, site-level metadata, enabling consistent and scalable generation of fully populated Zenodo records for each ES ID #. This separation ensured that participant privacy was preserved while maintaining the scientific and contextual integrity of each dataset.

8.2.4 Configuration of Project Metadata and Upload Settings (JSON)

Following the development of the HTML Zenodo record template and the Zenodo-specific metadata spreadsheets, JSON configuration files were created to define how these components are connected and used within the AZUS framework.

Two primary JSON configuration files, `project_config.json` and `config.json`, were used to specify project-level metadata, dataset locations, required metadata fields, output paths, and upload settings. Together, these files provided the instructions needed for AZUS to assemble standardized, site-level Zenodo records.

Within this system, the JSON configuration instructs AZUS to:

- use the HTML template as the basis for the Zenodo record description
- call site-specific values from the Zenodo-specific metadata spreadsheet
- populate variable fields in the template through dynamic substitution
- apply consistent project-level metadata across all records
- organize outputs into standardized site-level dataset structures, one per ES ID #

This configuration-driven design allowed the same template and spreadsheet structure to be reused across all records without hardcoding project-specific values into the processing script. It also ensured that record generation remained consistent, scalable, and reproducible across both eclipse campaigns.

In this way:

- **The template defines the structure and documentation framework**
- **The metadata spreadsheet supplies the site-specific values**
- **The JSON configuration defines how those values are applied and executed through AZUS**

Together, these components form an integrated preparation system that enables the consistent generation of fully populated Zenodo records for each ES ID #.

This configuration-driven design ensures that changes to dataset structure, metadata, or upload behavior can be implemented by modifying JSON configuration files rather than altering the underlying Python code, supporting scalability, reuse, and adaptation to other projects.

In addition to standard Zenodo metadata fields, Eclipse Soundscapes defined and consistently applied a controlled set of subject labels across all records within the Zenodo community. These subject labels, which are case-sensitive, were included as variable metadata elements and standardized across datasets to support consistent categorization.

For example, audio datasets were labeled by eclipse event (e.g., “2023 Annular Eclipse” and “2024 Total Solar Eclipse”), by site type or partner designation (e.g., “NPS Collection Site”), and by analysis relevance (e.g., “ES Data Analysis Site”).

This approach enables users to filter and sort records within the Eclipse Soundscapes Zenodo Community by subject, improving discoverability and navigation across the dataset collection.

8.2.5 Automated Dataset Assembly (AZUS Execution)

Following the configuration of templates, metadata spreadsheets, and JSON settings, dataset preparation and publication were executed using the Automated Zenodo Upload Software (AZUS).

AZUS is a command-line tool developed by the Eclipse Soundscapes team to automate the batch upload of structured datasets to Zenodo. Built on Zenodo’s Open API, AZUS streamlines the full data publishing process, including metadata assignment, licensing, file upload, and DOI generation. This system enabled the efficient archiving of hundreds of large, site-level datasets that would not have been feasible through Zenodo’s manual, one-record-at-a-time interface.

Within the Stage 3 workflow, AZUS serves as the execution layer that applies the configuration defined in Section 8.2.4 to generate and publish complete Zenodo records for each ES ID #.

Specifically, AZUS:

- reads configuration settings defined in the JSON files
- retrieves site-specific metadata from the Zenodo metadata spreadsheets
- populates the HTML Zenodo record template with site-level values
- assembles complete, site-level datasets (one per ES ID #)
- uploads files and metadata to Zenodo using the Open API
- assigns licensing and project-level metadata consistently across records
- submits each dataset to the Eclipse Soundscapes Zenodo Community

- publishes records and registers Digital Object Identifiers (DOIs)

This process follows a structured API-driven workflow in which each dataset progresses through:

1. Metadata and file preparation (completed before upload)
2. Record creation via Zenodo API (draft record creation)
3. File upload and metadata assignment
4. Submission to the ES Zenodo Community
5. Final publication and DOI generation

By automating these steps, AZUS ensures that all records are created using consistent metadata, standardized documentation, and uniform organizational structure.

AZUS is designed to:

- automate repetitive upload tasks for large volumes of data
- ensure each dataset includes standardized metadata and documentation
- eliminate Zenodo's manual one-record-at-a-time upload limitation
- support open access, citation, and long-term data preservation

AZUS includes a command-line interface with a dry-run mode that validates configuration, metadata, and file paths prior to initiating uploads, ensuring that datasets are fully prepared before interacting with the Zenodo API (see Appendix 1 for command-line usage and execution commands).

Originally developed to support the Eclipse Soundscapes project's goal of open, long-term data sharing, AZUS represents a reusable infrastructure for participatory science data stewardship. The software aligns with NASA's Open Science policies by enabling scalable, transparent, and reproducible data publication workflows.

AZUS is released as open-source software on GitHub under a BSD 3-Clause License, with documentation and example configurations to support adaptation by other research and citizen science projects.

Additional implementation details, including command-line execution, validation, and logging behavior, are provided in Appendix 2.

8.2.6 Output Packaging and Site-Level Record Structure

As part of the execution process, each site-level dataset was organized into a standardized structure to support accessibility, reproducibility, and long-term reuse.

The `prepare_dataset.py` script creates a staging directory for each site (e.g., `ES ID_XXX_Staging/`), which contains the complete dataset package, including the ZIP archive and all companion files prepared for upload. Within this staging process, the script performs a structured sequence of operations, including metadata extraction, ZIP archive creation, site-level metadata file generation, template population, documentation conversion, integrity hashing, and final dataset assembly. These prepared datasets are then used by AZUS for upload to Zenodo.

For each ES ID #, AZUS produced a complete dataset consisting of two coordinated components:

1. Site-Specific ZIP Archive (Primary Data Package)

- Raw audio files (WAV), preserved in original format
- AudioMoth configuration file (CONFIG.TXT)

- Included companion files necessary for data completeness

This ZIP archive serves as the authoritative package of raw data associated with each recording site.

2. Standalone Documentation and Metadata Files

- README files (.html and .md) generated from the Zenodo template
- Site-level metadata file (total_eclipse_data.csv)
- File manifest (file_list.csv) with SHA-512 hashes
- Data dictionaries, licensing information, and supporting documentation

These files are provided outside of the ZIP archive to ensure that key contextual and descriptive information is immediately accessible without requiring extraction of the primary data package.

Each dataset is organized around a single ES ID #, which functions as the unifying identifier linking all files, metadata, and documentation associated with a specific recording site. A site-level metadata file is generated for each ES ID # (e.g., a single-row CSV derived from the master metadata spreadsheet), and an upload manifest is created to support the structured transfer of files to Zenodo.

This packaging approach ensures that each dataset is:

- self-contained and fully documented
- structured for both human interpretation and machine processing
- verifiable through file-level integrity checks
- consistent across all sites and both eclipse campaigns

8.2.7 Final Output: Zenodo Records

The final output of Stage 3 is a collection of site-level Zenodo records, with one record created for each ES ID #.

Each record:

- contains the complete packaged dataset described in Section 8.2.6
- includes structured metadata derived from the Zenodo-specific metadata spreadsheets
- presents a fully populated, human-readable description generated from the HTML template
- is assigned a unique Digital Object Identifier (DOI) upon publication
- is organized within the Eclipse Soundscapes Zenodo Community

This structure ensures that each ES ID # corresponds to a single, independently accessible dataset that can be cited, downloaded, and reused without requiring reference to other records.

By organizing data at the site level and publishing each dataset with a DOI, the Eclipse Soundscapes project supports:

- long-term open access and preservation
- dataset-level citation and attribution
- transparency and reproducibility of data collection and processing methods
- scalable reuse of data across research, education, and public engagement contexts

Together, these records represent the final, publicly accessible output of the Stage 3 data preparation and publishing workflow.

9. DOI Creation and Versioning

Each Eclipse Soundscapes dataset is assigned a unique, data site-specific Digital Object Identifier (DOI) through Zenodo's integrated DOI minting service at the time of publication. These DOIs follow the pattern 10.5281/zenodo.XXXXXXX and provide a persistent, globally resolvable link to each dataset.

DOIs ensure that each site-level dataset can be independently cited, accessed, and reused. Once assigned, a DOI remains permanently associated with its dataset, regardless of future URL changes, supporting long-term accessibility and integration within the broader research ecosystem.

Zenodo also supports versioning of published records. If a dataset needs to be corrected or updated after publication, a new version can be created under the same DOI, which always resolves to the most recent version. Each version is also assigned its own version-specific DOI, ensuring previous published versions remain accessible and citable.

This versioning structure supports both:

- **reproducibility**, by preserving access to earlier dataset versions
- **data integrity**, by allowing updates without overwriting original records

This DOI and versioning framework aligns with FAIR data principles by ensuring that datasets are findable, accessible, and reusable over time.

Standardized subject labels and resource-type metadata further support discovery by enabling filtering and categorization within the Zenodo Community interface. For example, users can filter audio dataset records by eclipse event year (e.g., “2024 Total Solar Eclipse”) or by site designation (e.g., “ES Data Analysis Site”) to quickly locate relevant records.

10. Upload Verification and Integrity

As a final stage of the Stage 3 workflow, multiple verification mechanisms are used to ensure that all published records are complete, accurate, and uncorrupted.

10.1 SHA-512 file hashing

Every file's SHA-512 hash is computed during dataset preparation and recorded in *file_list.csv*. Researchers who download an ES dataset can recompute hashes locally and compare them against the manifest to verify that no data corruption occurred during transfer, storage, or decompression.

10.2 Upload Tracking and Result Logging

During and after upload execution, AZUS logs the outcome of every upload attempt. Successful uploads are appended to *Records/successful_results.csv* with the ES ID #, Zenodo record ID, DOI, upload timestamp, and API response details. Failed uploads are logged to *Records/failed_results.csv* with error type and message. An UploadTracker class maintains an

in-memory set of previously uploaded ES ID #s and skips them during batch runs, preventing duplicate uploads.

10.3 Metadata Completeness

During data preparation, before each upload, the pipeline validates that all required metadata fields are present in the collector CSV file. The expected headers are defined in *project_config.json* and checked at CSV file parse time. Missing or malformed fields cause an immediate, descriptive error rather than a partial upload. The `--dry-run` flag provides a safe way to validate the entire batch configuration before any API calls are made.

10.4 Request Logging

For each upload, AZUS writes the complete JSON payload sent to the Zenodo API and the API response to a per-ES ID # request log file (*ES ID_XXX_request_log.json*). These logs support post-upload auditing and debugging.

Together, these verification mechanisms ensure that all published datasets are complete, internally consistent, and reproducible, supporting reliable reuse within the broader scientific community.

11. Validation and Error Handling

Throughout the Stage 3 workflow, a series of validation and control points make sure that datasets meet all structural and metadata requirements before proceeding through preparation and upload.

11.1 Metadata Completeness Review

At dataset preparation time, the collector CSV file is parsed and validated against the schema defined in *project_config.json*. If required columns are missing or a row for the target ES ID # does not exist, the script exits with a descriptive error. This fail-fast behavior ensures that incomplete records are never packaged or uploaded.

11.2 ES ID # Alignment Validation

The ES ID # is extracted from the source folder name and matched against the master data collectors spreadsheet to ensure a valid site-level record exists. If the ES ID # in the folder name does not correspond to any row in the spreadsheet, preparation halts.

11.3 Manual Intervention for Flagged Inconsistencies

When automated checks detect issues (such as *CONFIG.TXT* not found, unexpected file counts, or API errors during upload), the pipeline treats these as validation failures, logs warnings, and halts rather than silently continuing. The operator reviews the log, resolves the issue manually, and re-runs. Failed uploads are tracked separately and do not affect successfully uploaded records in the same batch.

11.4 Dry-Run Mode

Both the preparation and upload scripts support dry-run execution. The upload script's `--dry-run` flag validates all configuration, CSV files, credential availability, and file paths without creating any Zenodo records. This provides an additional validation layer, allowing operators to verify an entire batch configuration before committing to live API operations.

Together, these validation and control mechanisms ensure that errors are detected early, prevent incomplete or inconsistent datasets from being published, and support reliable, repeatable execution of the Stage 3 workflow.

12. What These Procedures Support

The Stage 3 workflow and procedures described in this document collectively support the following outcomes:

Long-term preservation of raw eclipse audio data. By depositing datasets in Zenodo, a CERN-hosted repository with long-term preservation commitments, ES audio data is safeguarded against loss due to equipment failure, organizational changes, or funding expiration.

Public accessibility of site-level datasets. All records are published as open access with a Creative Commons license, enabling researchers, educators, and the public to freely access, download, and reuse the data.

Dataset traceability without publication of participant identities. The ES ID # system makes it possible for each dataset to be traced to its collection site and associated eclipse parameters while protecting volunteer privacy.

Citability through persistent DOI assignment. data site DOIs make each dataset independently citable, supporting proper attribution.

Reproducibility and reuse of archival audio data by independent researchers. The combination of unmodified raw audio, structured metadata, data dictionaries, and cryptographic hashes enables independent verification, reproducibility, and extension of the ES datasets.

Together, these outcomes demonstrate how a structured, configuration-driven data pipeline can support scalable, transparent, and reusable open science data sharing.

13. Open-Source Tooling and Generalization

The design and implementation of the Stage 3 workflow extend beyond the Eclipse Soundscapes project and were intentionally developed to support reuse by other research and participatory science efforts.

AZUS was designed to enable scalable, repeatable dataset publication workflows while minimizing the need for custom code. Project-specific elements, including metadata, citations,

file structures, and upload settings, are defined through human-readable CSV and JSON configuration files rather than embedded directly in Python code. This configuration-driven approach allows the system to be adapted for new projects with minimal modification.

The AZUS source code is publicly available on the ARISA Lab GitHub repository: github.com/ARISA-Lab-LLC/ESCSP. The codebase is released under the BSD 3-Clause license. This open-source distribution supports transparency, reproducibility, and community-driven adaptation of the workflow.

During active project operations, only high-level public descriptions of archival practices were shared publicly. This document provides formal technical documentation of the Zenodo integration and upload pipeline, contributing to broader open-science infrastructure and enabling other teams to implement similar data publication workflows.

14. Alignment with FAIR Principles

The Eclipse Soundscapes Stage 3 data workflow aligns with the FAIR data principles (Findable, Accessible, Interoperable, and Reusable), a widely adopted framework for maximizing the long-term value and usability of research data (go-fair.org/fair-principles/). While FAIR was not explicitly adopted during the initial project design, the development of the AZUS-based data pipeline resulted in systematic alignment with FAIR principles across all published datasets.

14.1 Findable

Making ES data findable required that every site-level dataset have a persistent, globally unique identifier. The ES ID # system provided internal identification; Zenodo's DOI assignment provided the public, citable identifier. Each ES ID # record became a separate Zenodo record with its own DOI, metadata, and structured description. This granularity of one record per recording site makes it possible for a future researcher to discover and cite a specific location's data independently or the entire dataset as a whole.

14.2 Accessible

The Accessibility dimension of FAIR is about ensuring that data, once found, can actually be retrieved. Zenodo is one of the very few platforms that can provide free, permanent, unrestricted data downloads at the scale required by the ES archive. Zenodo provides free, long-term, unrestricted access to all datasets, ensuring that data can be retrieved without institutional or financial barriers.

14.3 Interoperable

Making the data interoperable required significant additional work that had not been part of the original data pipeline plan. Interoperability means that a future researcher using different tools or working in a different domain can understand and work with the data without needing to contact the original team. The following were created specifically to meet this requirement:

- Machine and human-readable data dictionaries for each file type in the archive (WAV, CONFIG.TXT, file list, etc.)
- File manifests with SHA-512 cryptographic hashes, so a user can verify that downloaded files have not been corrupted or altered
- Standardized metadata schemas are applied consistently across all records

- The Stage documents themselves, which serve as provenance records explaining how data was collected, processed, and published

14.4 Reusable

Reusability is the goal all four FAIR principles ultimately serve. For the ES data archive, reusability is supported by the CC BY-SA 4.0 open license applied to all records, by the comprehensive provenance documentation in the Stage documents, by the structured metadata in the Zenodo Form Spreadsheets, and by the open-source release of AZUS itself.

Importantly, the ARISA Lab and Eclipse Soundscapes team identified that the broader citizen science, participatory science, and community science communities share the same need that created AZUS: a reliable, configuration-driven tool for batch-uploading structured datasets to Zenodo, without requiring that every project team have a professional software developer on staff.

AZUS is being developed for open-source release so that future projects can benefit from the infrastructure investment that the ES-CSP made.

15. Glossary

This glossary is shared across the Eclipse Soundscapes Stage 0–3 data management documents and defines terminology used throughout the ES data lifecycle workflow.

Term	Definition
AME / ES AMES	ES AudioMoth Metadata Extractor Suite. A software tool used to extract and evaluate embedded AudioMoth metadata, including timestamps, serial numbers, firmware information, and recording details to support validation and provenance workflows.
annular solar eclipse	when the Moon blocks the center of the Sun from view. The Moon appears to be too small to completely block the Sun.
API	Application Programming Interface. A structured method that allows software systems to communicate programmatically with external platforms or services.
Arbimon	An acoustic data platform originally considered by Eclipse Soundscapes for data storage, browsing, and analysis prior to the project's transition to Zenodo.
AudioMoth	An open-source, low-cost acoustic recording device used by ES volunteers to capture environmental soundscapes during eclipse events.
AZUS	Automated Zenodo Upload Software. A configuration-driven, open-source software pipeline developed by ARISA Lab to prepare, package, document, validate, and upload Eclipse Soundscapes datasets to Zenodo.
batch processing	A workflow approach that processes large groups of datasets simultaneously rather than handling one dataset at a time.
CONFIG.TXT	An AudioMoth-generated configuration file containing device settings such as sample rate, gain, firmware version, serial number, recording schedule, temperature, and battery state.
CSV	Comma-Separated Values. A simple, human- and machine-readable text file format used for spreadsheets and structured tabular data.
data dictionary	A structured reference file defining variable names, descriptions, data types, units, sources, code meanings, and notes associated with a dataset or metadata file.
dataset DOI / data site DOI	A persistent Digital Object Identifier assigned to a published Zenodo dataset record, allowing the dataset to be cited, accessed, and reused over time.

dry-run mode	A validation mode used by AZUS to verify configuration settings, metadata, credentials, and file paths before live uploads occur.
eclipse contact times	The calculated timing of eclipse phases for a specific site, including first contact, second contact, maximum eclipse, third contact, and fourth contact.
eclipse maximum	The moment of greatest solar obscuration at a specific location during a solar eclipse.
eclipse percent	The percentage of solar obscuration at a specific location.
ES ID #	Eclipse Soundscapes Identification Number. A unique numeric identifier assigned to each recording site, allowing datasets to be publicly shared without exposing participant identity.
ES WAVES	Eclipse Soundscapes WAV Audio Validation & Extraction Suite. A software system that ingests audio data from multiple microSD cards simultaneously, validates WAV files, extracts metadata, organizes datasets, and supports downstream analysis workflows.
FAIR principles	A framework for scientific data stewardship emphasizing that datasets should be Findable, Accessible, Interoperable, and Reusable.
file_list.csv	A manifest file included in each Zenodo record that lists every file, file size, file type, description, SHA-512 hash, and associated data dictionary.
first contact	solar eclipse Start (First Contact). The time at which the Moon first begins to obscure the Sun.
fourth contact	Eclipse End (Fourth Contact) The time at which the Moon completely stops blocking the Sun from view.
Hash / SHA-512 Hash	A cryptographic fingerprint used to verify file integrity. ES used SHA-512 hashes to confirm that files were not altered or corrupted during storage, transfer, or download.
Human-In-The-Loop review	A workflow step requiring manual human review and interpretation when automated validation systems identify incomplete, inconsistent, or ambiguous data conditions.
HTML README template	A reusable template used to generate standardized, site-specific Zenodo record descriptions and README documentation through automated metadata substitution.

InvenioRDM	The open-source digital repository framework underlying Zenodo. AZUS communicates with Zenodo through the InvenioRDM REST API.
JBOD	“Just a Bunch of Disks.” A direct-attached storage system consisting of multiple hard drives used for scalable local data storage.
JSON configuration file	A human-readable configuration file used by AZUS to define project metadata, upload settings, templates, required metadata fields, file paths, and workflow behavior.
Level 0 data	Raw, unprocessed data preserved in its original form without timestamp correction, resampling, filtering, or modification.
metadata	Structured descriptive information about data, including location, timestamps, eclipse parameters, device information, and processing documentation.
metadata reconciliation	The process of comparing and integrating metadata from multiple sources to resolve inconsistencies and establish validated site-level records.
microSD Card	A removable flash memory storage device used within AudioMoth recorders to store WAV audio recordings and configuration files.
participatory science	Scientific research conducted with participation from members of the public contributing observations, data collection, or analysis activities.
PII	Personally Identifiable Information. Protected personal information that could uniquely identify an individual participant.
provenance	Documentation describing the origin, processing history, validation status, and stewardship history of a dataset.
README	Human-readable documentation accompanying a dataset that explains its contents, structure, collection methods, metadata, and usage guidelines.
related identifiers	Structured citation metadata linking Zenodo records to related datasets, publications, software repositories, reports, or videos.
REST API	A web-based application programming interface that allows software systems to communicate using standardized HTTP requests.
scalability	The ability of a workflow, infrastructure system, or software tool to efficiently support increasing amounts of data, users, or processing demand.
second contact	Totality Start (for sites within the path of totality). The time at which totality begins during a total solar eclipse.

site-level dataset	A complete dataset associated with a single Eclipse Soundscapes recording location identified by one ES ID #.
staging directory	A temporary preparation directory created during AZUS processing that contains the ZIP archive and all companion files ready for upload to Zenodo.
subject labels	Standardized Zenodo metadata tags used to categorize datasets by eclipse event, analysis relevance, site type, or project designation.
template substitution	An automated process in which variable placeholders within templates are replaced with site-specific metadata values during dataset generation.

16. References and Related Resources

These references and related resources are shared across the Eclipse Soundscapes (ES) Stage 0–3 data management documents and provide supporting literature, technical resources, software references, infrastructure documentation, and related materials referenced throughout the ES data lifecycle workflow.

Core Eclipse Soundscapes Resources

Eclipse Soundscapes Zenodo Community

Public archive of Eclipse Soundscapes datasets, documentation, and related resources.

<https://zenodo.org/communities/eclipsesoundscapes>

ARISA Lab Eclipse Soundscapes GitHub Repository

Open-source software, workflows, processing tools, and supporting documentation for the Eclipse Soundscapes project. <https://github.com/ARISA-Lab-LLC/ESCSP>

Eclipse Soundscapes Technical Workflow References

Severino, M., & Winter, H. (2026). Eclipse Soundscapes Data Collector Role Training and Implementation Manual (2023–2024). Zenodo. <https://doi.org/10.5281/zenodo.18623443>

Severino, M., & Winter, H. T. Eclipse Soundscapes Data Management: Pre-Eclipse Infrastructure and Deployment Preparation (Stage 0). Zenodo.

Severino, M., & Winter, H. T. Eclipse Soundscapes Data Management: Receipt, Sorting, & Metadata Organization (Stage 1). Zenodo. <https://doi.org/10.5281/zenodo.19471425>

Severino, M., & Winter, H. T. Eclipse Soundscapes Data Management: Data Processing (Stage 2). Zenodo. <https://doi.org/10.5281/zenodo.18683402>

Winter, H. T., & Severino, M. Eclipse Soundscapes Data Management: Public Data Sharing (Stage 3). Zenodo. <https://doi.org/10.5281/zenodo.18683437>

Eclipse Timing and Astronomical Resources

NASA Goddard Space Flight Center Eclipse Web Site

Eclipse timing predictions and eclipse path calculations courtesy of Fred Espenak, NASA/GSFC. <https://eclipse.gsfc.nasa.gov/eclipse.html>

Wheeler, W. M., et al. (1935). Animal Behavior During a Total Solar Eclipse. Boston Society of Natural History.

AudioMoth and Acoustic Recording Resources and References

AudioMoth Operation Manual. (2022). Open Acoustic Devices.

theteam@openacousticdevices.info. Retrieved from:

<https://www.openacousticdevices.info/applications>

Open Acoustic Devices (AudioMoth)

AudioMoth device documentation, firmware information, and technical specifications.

<https://www.openacousticdevices.info/>

Hill, A. P., Prince, P., Piña Covarrubias, E., Doncaster, C. P., Snaddon, J. L., & Rogers, A. (2017). AudioMoth: Evaluation of a smart open acoustic device for monitoring biodiversity and the environment. *Methods in Ecology and Evolution*, 9(5), 1199–1211.

<https://doi.org/10.1111/2041-210X.12955>

Hill, A. P., Prince, P., Snaddon, J. L., Doncaster, C. P., & Rogers, A. (2019). AudioMoth: A low-cost acoustic device for monitoring biodiversity and the environment. *HardwareX*, 6, e00073. <https://doi.org/10.1016/j.ohx.2019.e00073>

Darras, K., Kolbrek, B., Knorr, A., Meyer, V., & Zippert, M. (2019). Assembling cheap, high-performance microphones for recording terrestrial wildlife: the Sonitor system [version 2; peer review: 3 approved]. *F1000Research*, 7, 1984. <https://doi.org/10.12688/f1000research.17511.2>

Soundscape Ecology and Acoustic Science References

Pijanowski, B. C., Villanueva-Rivera, L. J., Dumyahn, S. L., Farina, A., Krause, B. L., Napoletano, B. M., Gage, S. H., & Pieretti, N. (2011). Soundscape ecology: The science of sound in the landscape. *BioScience*, 61(3), 203–216. <https://doi.org/10.1525/bio.2011.61.3.6>

Open Science and Repository Infrastructure Resources

Zenodo

General-purpose open-access repository operated by CERN and OpenAIRE for long-term data preservation and DOI assignment. <https://zenodo.org>

FAIR Principles

Findable, Accessible, Interoperable, and Reusable data stewardship principles.

<https://www.go-fair.org/fair-principles/>

InvenioRDM

Open-source repository framework underlying Zenodo and used by AZUS through the REST API. <https://inveniordm.docs.cern.ch/>

Zenodo REST API Documentation

Documentation for Zenodo's Open API used by AZUS for automated dataset publication workflows.

<https://developers.zenodo.org/>

Arslan, R. C. (2019). How to Automatically Document Data With the Codebook Package to Facilitate Data Reuse. *Advances in Methods and Practices in Psychological Science*, 2(2), 169–187. <https://doi.org/10.1177/2515245919838783>

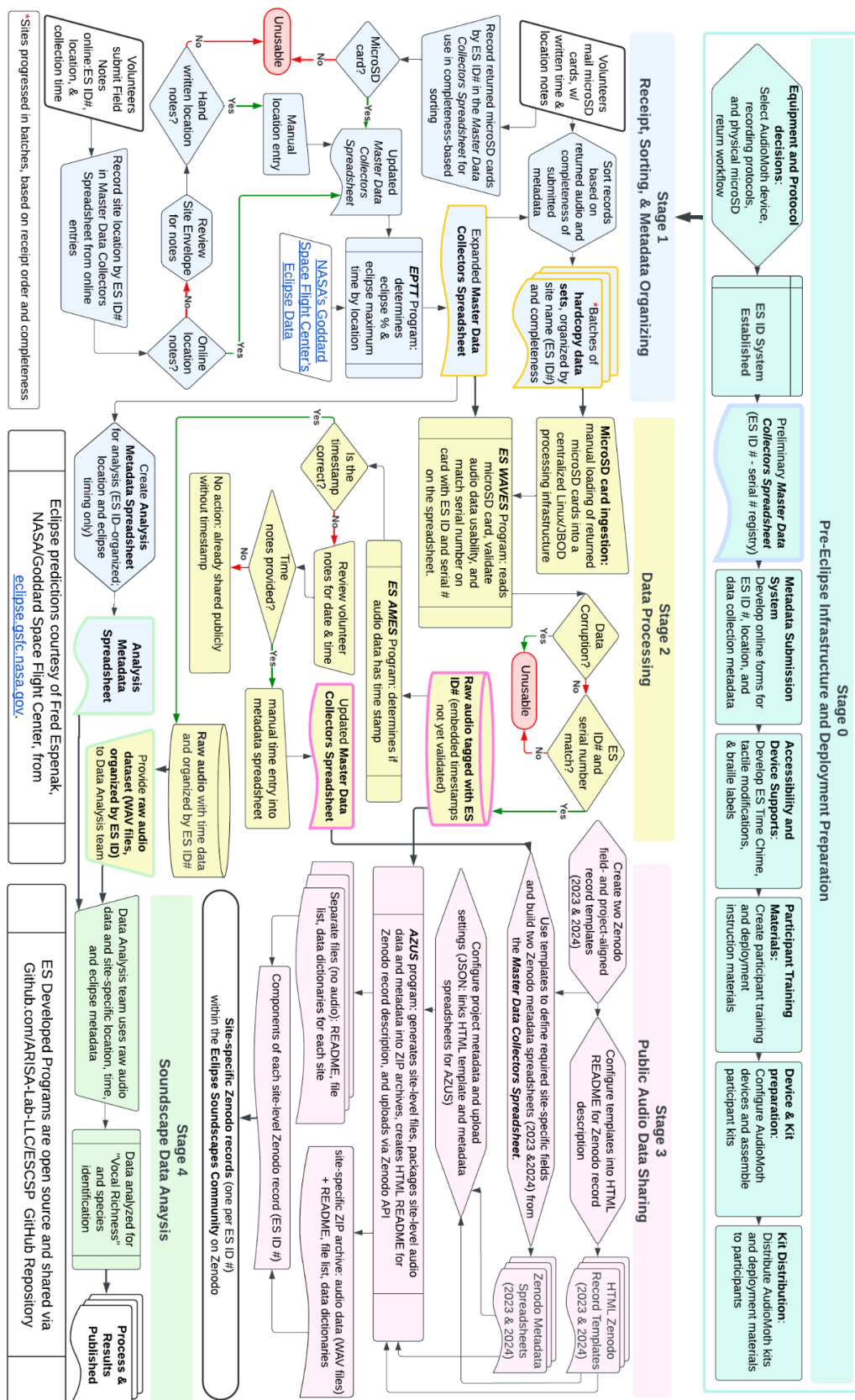
Barnes, N. (2010). Publish your computer code: it is good enough. *Nature*, 467, 753. <https://doi.org/10.1038/467753a>

Supporting Technical and Infrastructure References

US Household Average Internet Speeds: Average U.S. Internet Speeds Over Time. Husain Sumra (June 5, 2025).

<https://www.ooma.com/blog/average-us-internet-speeds-over-time/>

Appendix 1: ES Operational Infrastructure and Data Stewardship Workflows



Appendix 2: Upload Command Line Interface (CLI)

To run a batch upload the following AZUS commands are run:

```
source Resources/set_env.sh          # Load Zenodo API credentials
python standalone_tasks.py --dry-run  # Validate configuration
python standalone_tasks.py           # Execute upload
```

The `--dry-run` flag validates all configuration, CSV files, and file paths without contacting the Zenodo API. When running without `--dry-run`, the script displays a confirmation prompt before proceeding. When these commands are run AZUS displays the current status of each upload with timestamps given at each step. Logs of successful and unsuccessful uploads are written to the *Records/* subdirectory of the AZUS codebase.

Appendix 3: AZUS Command-Line Interface and Execution

A3.1 Overview

This appendix provides technical details on the command-line interface (CLI) and execution workflow of the Automated Zenodo Upload Software (AZUS). These details supplement the Stage 3 workflow described in Section 8 and are intended to support reproducibility and reuse of the AZUS system.

A3.2 Execution Environment and Setup

AZUS is executed in a command-line environment with access to Zenodo API credentials. Environment variables are configured prior to execution to enable authenticated communication with the Zenodo API.

source: Resources/set_env.sh # Load Zenodo API credentials

A3.3 Batch Upload Execution Commands

```
python standalone_tasks.py --dry-run # Validate configuration and inputs
python standalone_tasks.py # Execute batch upload
```

The `--dry-run` flag performs validation of configuration files, metadata spreadsheets, and file paths without initiating communication with the Zenodo API. This step ensures that all datasets are correctly prepared prior to upload.

A3.4 Upload Workflow Behavior

When executed, AZUS processes each dataset sequentially using a structured API-driven workflow:

1. Create draft record with metadata and community association
2. Upload dataset files (ZIP archive and companion files)
3. Submit record to the Eclipse Soundscapes Zenodo Community
4. Publish record and register DOI

A3.5 Validation and User Interaction

When running without the `--dry-run` flag, AZUS prompts the user for confirmation before initiating uploads. During execution, the system displays the status of each dataset upload, including timestamps for each step.

A3.6 Logging and Output Tracking

Logs of successful and unsuccessful uploads are written to the `Records/` subdirectory of the AZUS codebase. These logs provide a record of upload status, timing, and any errors encountered during execution.

A3.7 Configuration Dependence

All execution behavior is controlled through JSON configuration files. Changes to dataset structure, metadata inputs, or upload behavior can be implemented by modifying configuration files rather than altering the underlying Python code.